

Caio Collino Troiano
Thiago Pereira Lins da Silva
Vinicius Vianna Egydio de Carvalho

Análise de Desempenho - Computação Paralela

São Paulo
2026

Sumário

1	RESULTADOS E ANÁLISE	2
1.1	Objetivo	2
1.2	Resultados Obtidos	2
1.2.1	Tempo de Execução	2
1.2.2	Speedup	3
1.3	Análise dos Resultados	3
1.4	Análise de Eficiência	4
1.5	Escalabilidade	4
1.6	Limitações do Experimento	5
1.7	Conclusão	5

1 Resultados e Análise

1.1 Objetivo

O objetivo deste experimento foi avaliar o desempenho de uma implementação paralela em comparação com sua versão sequencial, analisando o impacto da quantidade de threads no tempo de execução, no speedup e na eficiência do algoritmo.

Os testes foram realizados para diferentes tamanhos de entrada ($N = 1000, 2000, 4000, 8000$ e 10000), utilizando 2, 4, 8 e 16 threads. Para cada configuração foram obtidos os tempos médios de execução, permitindo o cálculo do speedup e da eficiência.

1.2 Resultados Obtidos

1.2.1 Tempo de Execução

A Figura 1 apresenta a variação do tempo médio de execução da versão paralela em função do número de threads para cada tamanho de entrada analisado.

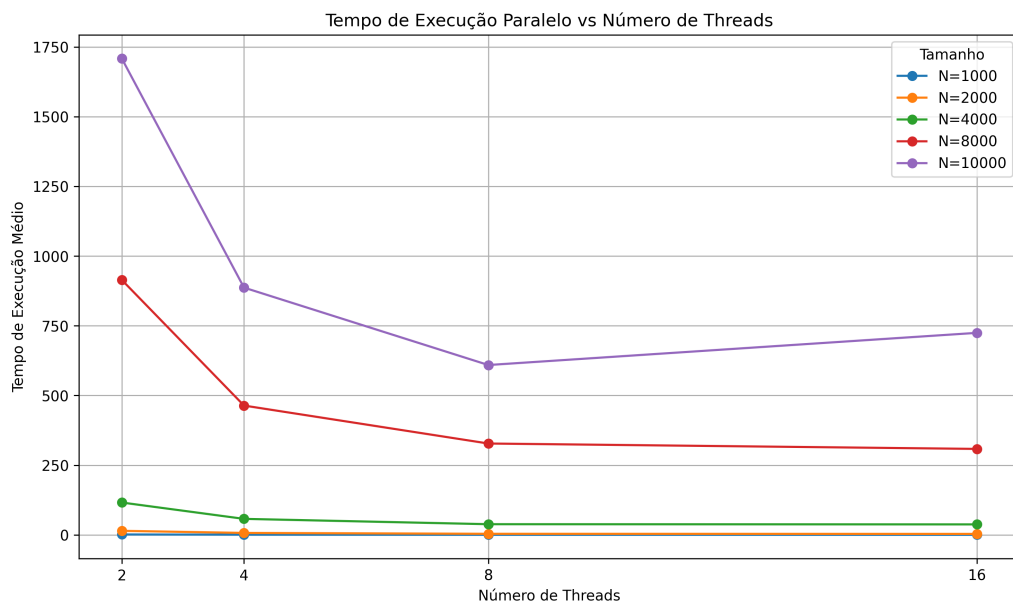


Figura 1 – Tempo médio de execução em função do número de threads.

Observa-se que o aumento do número de threads reduz significativamente o tempo de execução para todos os tamanhos de entrada. Entretanto, essa redução não ocorre de forma linear, especialmente quando a quantidade de threads ultrapassa oito unidades.

Os maiores ganhos são observados ao passar de 2 para 4 threads e de 4 para 8 threads. A partir desse ponto, os benefícios tornam-se progressivamente menores devido aos custos de sincronização, comunicação e gerenciamento das threads.

1.2.2 Speedup

A Figura 2 apresenta o speedup obtido para cada tamanho de entrada em comparação com a execução sequencial.

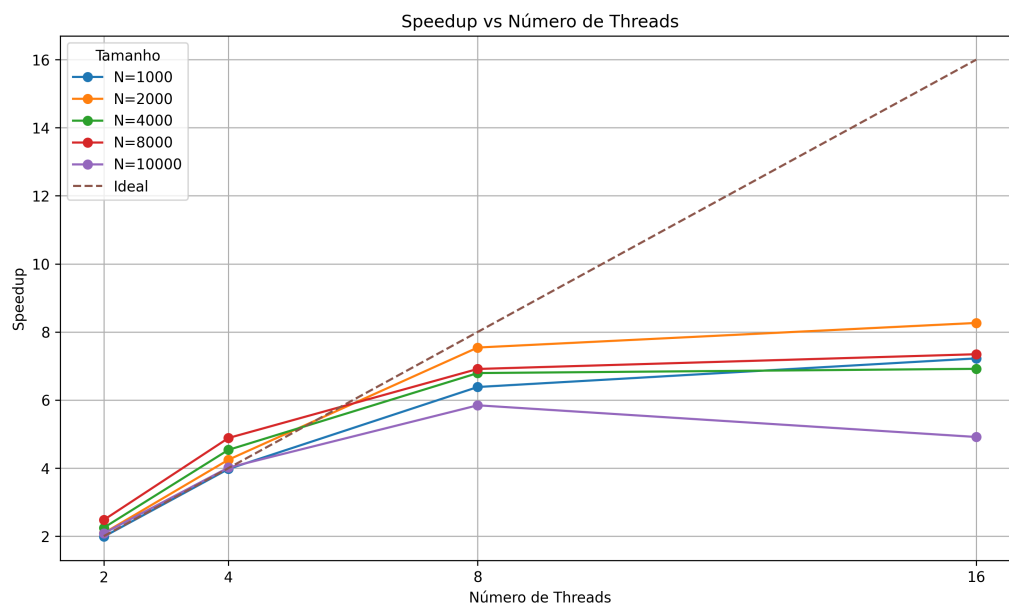


Figura 2 – Speedup em função do número de threads.

A linha tracejada presente no gráfico representa o speedup ideal, no qual o ganho de desempenho seria exatamente proporcional ao número de threads utilizadas.

Os resultados mostram que o algoritmo apresenta excelente escalabilidade até aproximadamente 8 threads. Para os tamanhos intermediários ($N = 2000, 4000$ e 8000), o speedup obtido aproxima-se significativamente do comportamento ideal.

O melhor resultado foi observado para $N = 2000$ utilizando 16 threads, alcançando um speedup de aproximadamente 8,26 vezes em relação à execução sequencial.

1.3 Análise dos Resultados

Os resultados evidenciam o impacto positivo da paralelização sobre o desempenho do algoritmo.

Para o menor conjunto de dados ($N = 1000$), o speedup cresce rapidamente até 8 threads, alcançando aproximadamente 6,38. Ao utilizar 16 threads, o ganho adicional é pequeno, atingindo cerca de 7,22. Esse comportamento sugere que os custos de gerenciamento das threads passam a representar uma parcela significativa do tempo total de execução.

À medida que o tamanho do problema aumenta, os benefícios da paralelização tornam-se mais evidentes. Para $N = 8000$, por exemplo, o tempo de execução foi reduzido de aproximadamente 2265 unidades de tempo na versão sequencial para cerca de 308 unidades utilizando 16 threads.

Esse comportamento está relacionado ao aumento da granularidade do trabalho. Problemas maiores oferecem mais operações independentes que podem ser distribuídas entre as threads, reduzindo proporcionalmente o impacto das sobrecargas associadas ao paralelismo.

Entretanto, observa-se que os ganhos deixam de crescer proporcionalmente ao número de threads disponíveis. Esse fenômeno está diretamente relacionado aos limites impostos pela Lei de Amdahl, segundo a qual a parcela sequencial do algoritmo passa a dominar a execução conforme o número de processadores aumenta.

1.4 Análise de Eficiência

A eficiência é calculada pela Equação 1.1:

$$E = \frac{S}{P} \quad (1.1)$$

onde:

- E representa a eficiência;
- S representa o speedup;
- P representa o número de threads.

Os resultados indicam eficiência próxima de 100

Em alguns casos, foram observadas eficiências ligeiramente superiores a 100

Com 8 threads, a eficiência permanece elevada, geralmente acima de 80

Já para 16 threads ocorre uma queda significativa da eficiência, variando entre aproximadamente 30

1.5 Escalabilidade

A análise dos gráficos demonstra que a implementação apresenta boa escalabilidade para problemas de médio e grande porte.

Nos menores tamanhos de entrada, o custo de criação e sincronização das threads reduz a efetividade do paralelismo. Em contrapartida, conforme o tamanho do problema cresce, esses custos tornam-se menos relevantes em relação ao volume total de processamento realizado.

Os resultados obtidos para os conjuntos de dados com $N = 4000$ e $N = 8000$ representam os cenários mais próximos do comportamento ideal, evidenciando que o algoritmo consegue aproveitar de forma eficiente os recursos computacionais disponíveis.

Entretanto, a redução do speedup observada para $N = 10000$ ao passar de 8 para 16 threads sugere a existência de gargalos relacionados à arquitetura de hardware, largura de banda de memória ou contenção entre threads.

1.6 Limitações do Experimento

Algumas limitações devem ser consideradas durante a análise dos resultados:

- Não foram consideradas métricas de utilização de CPU, memória ou cache.
- O experimento foi limitado a no máximo 16 threads.
- Não houve comparação entre diferentes estratégias de paralelização.
- Fatores externos, como processos concorrentes do sistema operacional, podem ter influenciado os tempos medidos.
- As características específicas do hardware utilizado não foram consideradas na análise quantitativa dos resultados.

1.7 Conclusão

Os resultados obtidos demonstram que a paralelização foi altamente eficaz para reduzir o tempo de execução do algoritmo.

Observou-se um crescimento consistente do speedup à medida que o número de threads aumentou, especialmente para problemas de maior dimensão. Os melhores resultados foram alcançados entre 8 e 16 threads, dependendo do tamanho da entrada.

A eficiência manteve-se elevada para até 8 threads, indicando excelente aproveitamento dos recursos disponíveis. Entretanto, a utilização de 16 threads evidenciou limitações inerentes à computação paralela, como custos de sincronização, contenção de recursos e aumento da complexidade de gerenciamento.

Conclui-se que a implementação paralela apresenta boa escalabilidade e oferece ganhos substanciais de desempenho para problemas computacionalmente intensivos. Contudo, existe um ponto de saturação a partir do qual o aumento do número de threads deixa de produzir benefícios proporcionais, reforçando a necessidade de identificar a configuração ótima para cada ambiente de execução.